

1 Problem 1

The gradient descent algorithm can also benefit from the use of multiple initial points (multi-start) to find local minima. Generate the gradient descent version with multiple starting points using CUDA.

For this exercise, we will use the following equation:

$$f(x) = x^2 + 10 * \sin(x) \quad (1)$$

$$f'(x) = 2x + 10 * \cos(x) \quad (2)$$

```
def f(x):
    return x**2 + 10*np.sin(x)
```

First, kernel definition:

```
mod = SourceModule("""
__global__ void multi_start_gd(float *points, float lr, int iterations, int n)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n){
        float x = points[idx];

        // gradient descent loop
        for (int i = 0; i < iterations; i++){
            // f(x) = x^2 + 10*sin(x)
            // f'(x) = 2*x + 10*cos(x)
            float gradient = 2.0f*x+10.0f*cosf(x);
            x = x - lr * gradient;
        }
        points[idx] = x; // overwrite local min found
    }
}""")
```

Define the starting points

```
N = 1024 #starting points
iterations = 1000
learning_rate = 0.01
start_points = np.random.uniform(-10,10,N).astype(np.float32)
res_points = start_points.copy()
```

```
gd_kernel = mod.get_function("multi_start_gd")
gd_kernel(
    drv.InOut(res_points),
    np.float32(learning_rate),
    np.int32(iterations),
    np.int32(N),
    block = (256,1,1),
    grid=(int(np.ceil(N/256)),1))
```

By running these specific cells, we obtain the following results:

```
[9]
✓ Os
print(f"first 5 starting points: {start_points[:5]}")
print(f"first 5 local minima found: {res_points[:5]}")
print(f"global minimum found: {np.min(res_points)}")

first 5 starting points: [ 4.809615  9.882414  9.4745   -0.7182164 -9.849012 ]
first 5 local minima found: [ 3.8374684  3.8374684  3.8374684 -1.3064395 -1.3064405]
global minimum found: -1.3064404726028442
```

Figure 1: Starting points, local minima, and global minima found.

```
[17]
✓ Os
best_x = res_points[np.argmin(f(res_points))]
print(f"Global Minimum found by GPU: x = {best_x:.4f}, f(x) = {f(best_x):.4f}")

Global Minimum found by GPU: x = -1.3064, f(x) = -7.9458
```

Figure 2: Global minimum found by GPU.

For global minimum found: -1.3064404 , $x \approx -1.31$, where $f(x) \approx -8.31$.

In addition, let's plot the results:

```
x_range = np.linspace(-10,10,1000)
plt.figure(figsize=(12,6))

plt.plot(x_range, f(x_range), label = 'Objective Function',
        color = 'black',
        alpha = 0.3)

plt.scatter(res_points, f(res_points),
        color='red',
        s=10,
        label='GPU Converged Points',
```

```
zorder=3)

plt.scatter(start_points[:50],f(start_points[:50]),
            color='blue',
            s=5,
            label='Sample Starting Positions',
            alpha=0.5)

plt.title(f"Multi-start Gradient Descent ({N} Threads)")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)
plt.show()
```

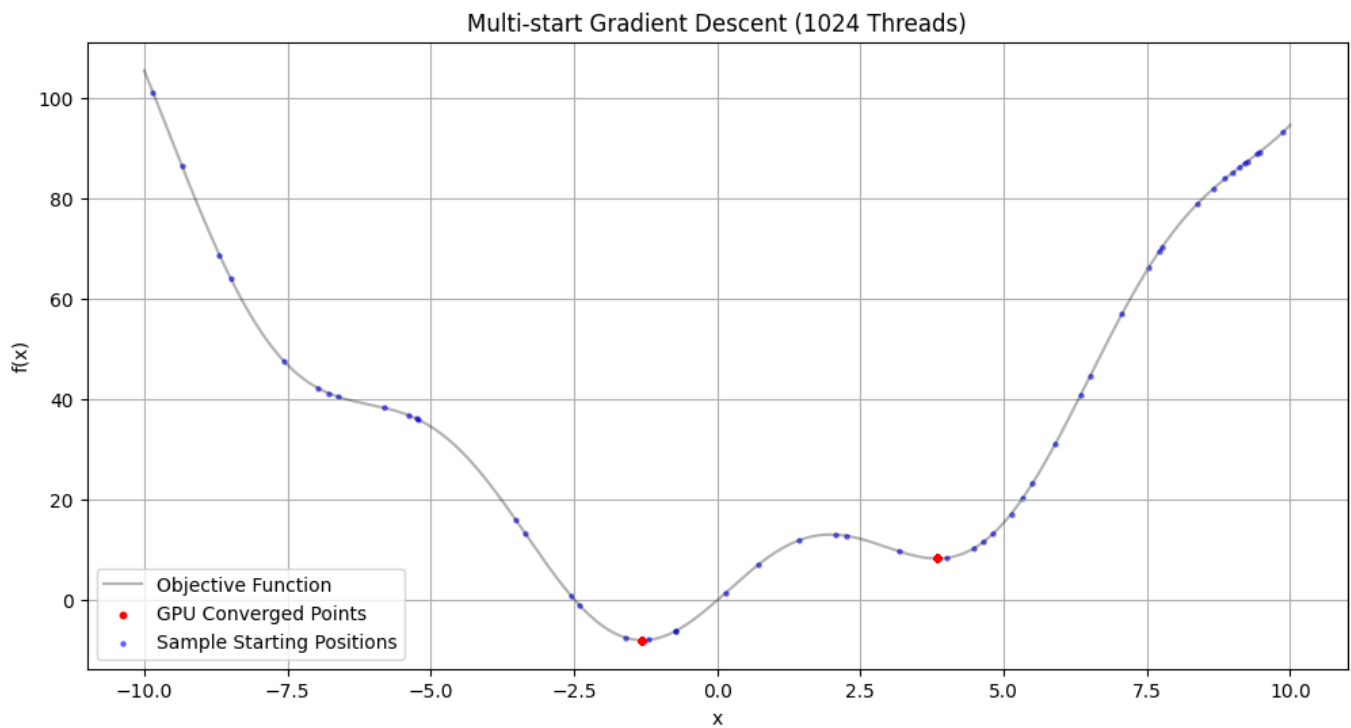


Figure 3: Gradient Descent result.

Homework 3:

2. Function (x, y)

$$\alpha = 0,2$$

$$(x_0, y_0) = (3, 5)$$

$$F(x, y) = x^4 - x^2 + y^2$$

↳ Minimize function

Analyze perf by 3 iterations

$$\nabla F(x, y) = (4x^3 - 2x, 2y)$$

$$g^0 = \nabla F(3, 5) = (102, 10)$$

$$\hookrightarrow x^{(1)} = x^0 - \alpha g^0$$

1st iteration

$$x^{(1)} = (3, 5) - 0,2(102, 10)$$

$$= (3, 5) - (20,4, 2)$$

$$x^{(1)} = (-17,4, 3)$$

$$\rightarrow \nabla F(-17,4, 3) = g^1$$

$$g^{(1)} = (4(-17,4)^3 - 2(-17,4), 2(3))$$

$$g^1 = (-21072.096 + 34.8, 6) \\ = (-21037.296, 6)$$

$$\hookrightarrow X^{(2)} = X^{(1)} - \alpha(g^{(1)})$$

$$= (-17.4, 3) - 0.2(-21037.296, 6)$$

$$= (-17.4, 3) - (-4207.4592, 1.2)$$

$$= (4190.059, 1.8) = X^{(2)}$$

$$g^{(2)} = (4(4190.059)^3 - 2(4190.059), \\ 2(1.8))$$

$$g^{(2)} = (294.3 \times 10^9, 3.6)$$

$$X^{(3)} = (4190.059, 1.8) - 0.2(294.3 \times 10^9, 3.6)$$

$$= (4190.059, 1.8) - (58.86 \times 10^9, 0.72)$$

$$X^{(3)} = (-58.859 \times 10^9, 1.08)$$